Geometric Graph Representation Learning via Maximizing Rate Reduction

Xiaotian Han
Texas A&M University, TX, USA
han@tamu.edu

Zhimeng Jiang
Texas A&M University, TX, USA
zhimengj@tamu.edu

Ninghao Liu
University of Georgia, GA, USA
ninghao.liu@uga.edu

Qingquan Song
LinkedIn, CA, USA
qsong@linkedin.com

Jundong Li
University of Virginia, VA, USA
jundong@virginia.edu

Xia Hu
Rice University, TX, USA
xia.hu@rice.edu

ABSTRACT

Learning discriminative node representations benefits various downstream tasks in graph analysis such as community detection and node classification. Existing graph representation learning methods (e.g., based on random walk and contrastive learning) are limited to maximizing the *local* similarity of connected nodes. Such pair-wise learning schemes could fail to capture the *global* distribution of representations, since it has no explicit constraints on the global geometric properties of representation space. To this end, we propose Geometric Graph Representation Learning (G^2R) to learn node representations in an unsupervised manner via maximizing rate reduction. In this way, G^2R maps nodes in distinct groups (implicitly stored in the adjacency matrix) into different subspaces, while each subspace is compact and different subspaces are dispersedly distributed. G^2R adopts a graph neural network as the encoder and maximizes the rate reduction with the adjacency matrix. Furthermore, we theoretically and empirically demonstrate that rate reduction maximization is equivalent to maximizing the principal angles between different subspaces. Experiments on real-world datasets show that G^2R outperforms various baselines on node classification and community detection tasks.

CCS CONCEPTS

• **Computing methodologies** → **Unsupervised learning**; **Neural networks**; • **Networks** → **Social media networks**.

KEYWORDS

Graph representation learning, Graph neural networks, Unsupervised learning, Rate reduction

ACM Reference Format:
Xiaotian Han, Zhimeng Jiang, Ninghao Liu, Qingquan Song, Jundong Li, and Xia Hu. 2022. Geometric Graph Representation Learning via Maximizing Rate Reduction. In *Proceedings of the ACM Web Conference 2022 (WWW '22), April 25–29, 2022, Virtual Event, Lyon, France*. ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3485447.3512170

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
WWW '22, April 25–29, 2022, Virtual Event, Lyon, France
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9096-5/22/04…$15.00
https://doi.org/10.1145/3485447.3512170

1 INTRODUCTION

Learning effective node representations [14] benefits various graph analytical tasks, such as social science [43], chemistry [6], and biology [49]. Recently, graph neural networks (GNNs) [41, 46] have become dominant technique to process graph-structured data, which typically need high-quality labels as supervision. However, acquiring labels for graphs could be time-consuming and unaffordable. The noise in labels will also negatively affect model training, thus limiting the performance of GNNs. In this regard, learning high-quality low-dimensional representations with GNNs in an unsupervised manner is essential for many downstream tasks. Recently, many research efforts have been devoted to learning node representations in an unsupervised manner. Most existing methods can be divided into two categories, including random walk based methods [13, 30] and contrastive learning methods [38, 44]. These methods learn node representations mainly through controlling the representation similarity of connected nodes. For example, DeepWalk [30] considers the similarity of nodes in the same context window of random walks. GRACE [48] uses contrastive learning to model the similarity of connected nodes with features. Such a pair-wise learning scheme encourages the *local* representation similarity between connected nodes, but could fail to capture the *global* distribution of node representations, since it does not directly specify the geometrical property of latent space. To bridge the gap, we propose to explicitly control the global geometrical discriminativeness of node representations instead of only enforce the local similarity of connected nodes. However, directly constraining the global geometric property of the representation space remains challenging due to the following reasons. First, it is difficult to measure the diversity of representations within the same group or across different groups, since the global information such as community distribution is not available in unsupervised settings. Pre-computed node clustering will not fully solve the problem, because there is no guarantee on the quality of resultant clusters, and it even introduces noisy supervised information. Second, it is hard to balance the global geometric property and local similarity, especially when considering the downstream tasks. Since the local similarity of connected nodes is crucial to the performance of downstream tasks, we need to control the global geometric property and local similarity simultaneously.

To address the above challenges, we propose Geometric Graph Representation Learning (G^2R) to learn node representations via maximizing coding rate reduction. First, we leverage the coding rate [45] to estimate the diversity of a set of node representations.

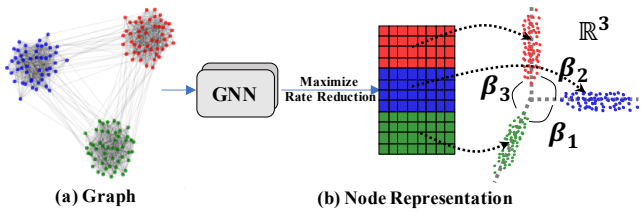


Figure 1: Overview of G^{2}R. It maps nodes in distinct groups (implicitly preserved in adjacency matrix) into different subspaces, while each subspace is compact and different subspaces are dispersedly distributed. Different colors indicate different subspaces and node groups.

A higher coding rate means representations are diversely spread in the latent space. Also, we define *rate reduction* as the difference of coding rates between representations of the entire nodes and each of the groups. Then, we maximize the rate reduction to learn geometrically discriminative representations. A higher rate reduction means node representations are close to each other within each group, while they are far away from each other across different groups. This can be achieved even without explicitly knowing the node-group assignments. We use graph neural networks as the encoder to generate node representations, and map the nodes in the same group into the identical latent subspace. Specifically, Figure 1 presents an intuitive overview of G^{2}R. The nodes in green, blue and red (Figure 1(a)) are projected to different subspaces (Figure 1(b)), and the difference between subspaces are maximized. The main contributions are summarized as follows:

- We propose a new objective for unsupervised graph learning via maximizing rate reduction, which encourages the encoder to learn discriminative node representations with only the adjacency matrix (Section 3).
- We provide theoretical justification for the proposed method from the perspective of maximizing the principal angles between different latent subspaces. (Section 4).
- Experimental results on synthetic graphs validate the theoretical results of the proposed method (Section 5.1). And the visualization results on real-world datasets also verify that resultant node representations are nearly orthogonal (Section 5.3).
- Experimental results on real-world datasets show that the node representations learned by G^{2}R are consistently competitive on the node classification and community detection tasks. Besides, G^{2}R achieves comparable performance to supervised baselines on node classification (Section 5.2 and 5.8).

2 PRELIMINARIES

In this section, we present essential preliminaries. First, we introduce the notations in this work. Then we introduce the idea of rate reduction for representation learning.

2.1 Notations

A graph is denoted as $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, where $\mathcal{V}$ is the node set and $\mathcal{E}$ is the edge set. The number of nodes is N. The adjacency matrix is denoted as $\mathbf{A} = [\mathbf{a}_1, \mathbf{a}_2, \cdots, \mathbf{a}_N] \in \mathbb{R}^{N\times N}$, where $\mathbf{a}_i$ is the neighbor indicator vector of node i. The feature matrix is $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_N] \in \mathbb{R}^{d_0\times N}$, where d_0 is the dimension of node features. A graph neural network encoder is denoted as $\mathrm{Enc}(\mathbf{A}, \mathbf{X})$, which transforms the nodes to representations $\mathbf{Z} = [\mathbf{z}_1, \mathbf{z}_2, \cdots, \mathbf{z}_N] \in \mathbb{R}^{d\times N}$, where d is the dimension of $\mathbf{z}_i$.

2.2 Representation Learning via Maximizing Rate Reduction

In this part, we introduce rate reduction [45], which was proposed to learn diverse and discriminative representations. The coding rate [21] is a metric in information theory to measure the compactness of representations over all data instances. A lower coding rate means more compact representations. Suppose a set of instances can be divided into multiple non-overlapping groups. Rate reduction measures the difference of coding rates between the entire dataset and the sum of that of all groups. Higher rate reduction implies more discriminative representation among different groups and more compact representation within the same group.

Representation Compactness for the Entire Dataset. Let $f(\cdot)$ denote the encoder, where the representation of a data instance $\mathbf{x}_i$ is $\mathbf{z}_i = f(\mathbf{x}_i) \in \mathbb{R}^d$. Given the representations $\mathbf{Z} = [\mathbf{z}_1, \mathbf{z}_2, \cdots, \mathbf{z}_N] \in \mathbb{R}^{d\times N}$ of all data instances, the coding rate is defined as the number of binary bits to encode $\mathbf{Z}$, which is estimated as below [21]:

$$R(\mathbf{Z}, \epsilon) \doteq \frac{1}{2}\log\det\left(\mathbf{I} + \frac{d}{N\epsilon^2}\mathbf{Z}\mathbf{Z}^\top\right), \tag{1}$$

where $\mathbf{I}$ is the identity matrix, N and d denote the length and dimension of learned representation $\mathbf{Z}$, and ϵ is the tolerated reconstruction error (usually set as a heuristic value 0.05).

Representation Compactness for Groups. Given $\mathbf{Z} = [\mathbf{z}_1, \mathbf{z}_2, \cdots, \mathbf{z}_N] \in \mathbb{R}^{d\times N}$, we assume the representations can be partitioned to K groups with a probability matrix $\pi \in \mathbb{R}^{N\times K}$. Here $\pi_{ik} \in [0, 1]$ indicates the probability of instance $\mathbf{x}_i$ assigned to the subset k, and $\sum_{k=1}^{K} \pi_{ik} = 1$ for any $i \in [N]$. We define the membership matrix for subset k as $\mathbf{\Pi}_k = diag[\pi_{1k}, \pi_{2k}, \cdots, \pi_{Nk}] \in \mathbb{R}^{N\times N}$, and the membership matrices for all groups are denoted as $\mathbf{\Pi} = \{\mathbf{\Pi}_k | k = [K]\}$. Thus, the coding rate for the entire dataset is equal to the summation of coding rate for each subset:

$$R^c(\mathbf{Z}, \epsilon|\mathbf{\Pi}) \doteq \sum_{k=1}^{K} \frac{tr(\mathbf{\Pi}_k)}{2\cdot N}\cdot\log\det\left(\mathbf{I} + \frac{d}{tr(\mathbf{\Pi}_k)\epsilon^2}\mathbf{Z}\mathbf{\Pi}_k\mathbf{Z}^\top\right). \tag{2}$$

Rate Reduction for Representation Learning. Intuitively, the learned representations should be diverse in order to distinguish instances from different groups. That is, *i) the coding rate for the entire dataset should be as large as possible to encourage diverse representations* ; *ii) the representations for different groups should span different subspaces and be compacted within a small volume for each subspace.* Therefore, a good representation achieves a *larger* rate reduction (i.e., difference between the coding rate for datasets and the summation of that for all groups):

$$\Delta R(\mathbf{Z}, \mathbf{\Pi}, \epsilon) = R(\mathbf{Z}, \epsilon) - R^c(\mathbf{Z}, \epsilon|\mathbf{\Pi}). \tag{3}$$

Note that the rate reduction is monotonic with respect to the norm of representation $\mathbf{Z}$. So we need to normalize the scale of the learned features, each $\mathbf{z}_i$ in $\mathbf{Z}$ is normalized in our case.

3 METHODOLOGY

In this section, we introduce our G^{2}R model based on rate reduction for unsupervised graph representation learning. Specifically, we first introduce how to compute the coding rate of node representations for the nodes in the whole graph and in each group, respectively. Then, we introduce how to incorporate rate reduction into the design of the learning objective and how to train G^{2}R.

3.1 Coding Rate of Node Representations

Our goal is to learn an *encoder* $\mathbf{Z} = \text{Enc}(\mathbf{A}, \mathbf{X}|\theta)$, which transforms the graph to the node representations, where $\mathbf{Z} \in \mathbb{R}^{d\times N}$ and θ is the encoder parameters to be optimized. The encoder in this work is instantiated as a graph neural network. The learned node representations will be used for various downstream applications, such as node classification and community detection.

3.1.1 Computing Coding Rate of Entire Node Representations. Let $\mathbf{Z} = [\mathbf{z}_1, \mathbf{z}_2, \cdots, \mathbf{z}_N] \in \mathbb{R}^{d\times N}$ be the node representations. We use coding rate to estimate the number of bits for representing $\mathbf{Z}$ within a specific tolerated reconstruction error ϵ. Therefore, in graph $\mathcal{G}$, the coding rate of node representations is $R_{\mathcal{G}}(\mathbf{Z}, \epsilon) = R(\mathbf{Z}, \epsilon)$ as defined in Equation 1. A larger $R_{\mathcal{G}}$ corresponds to more diverse representations across nodes, while a smaller $R_{\mathcal{G}}$ means a more compact representation distribution.

3.1.2 Computing Coding Rate for Groups. To enforce the connected nodes have the similar representations, we cast the *node i and its neighbors as a group* and then map them to identical subspace. To do this, we assemble the membership matrix based on the adjacency matrix. The adjacency matrix is $\mathbf{A} = [\mathbf{a}_1, \mathbf{a}_2, \cdots, \mathbf{a}_N] \in \mathbb{R}^{N\times N}$ where $\mathbf{a}_i \in \mathbb{R}^N$ is the neighbor indicator vector of node i. Then we assign membership matrix for the node group as $\mathbf{A}_i = diag(\mathbf{a}_i) \in \mathbb{R}^{N\times N}$. The coding rate for the group of node representations with membership matrix $\mathbf{A}_i$ is as follows:

$$R^c_{\mathcal{G}}(\mathbf{Z}, \epsilon|\mathbf{A}_i) \doteq \frac{\text{tr}(\mathbf{A}_i)}{2N} \cdot \log\det\left(\mathbf{I} + \frac{d}{\text{tr}(\mathbf{A}_i)\epsilon^2}\mathbf{Z}\mathbf{A}_i\mathbf{Z}^\top\right). \quad (4)$$

Thus for all nodes in the graph, the membership matrix set will be $\mathcal{A} = \{\mathbf{A}_i \in \mathbb{R}^{N\times N}, i \in [N]\}$. Since the $\sum_{i=1}^N \mathbf{A}_i = \mathbf{D}$, where $\mathbf{D} = diag(d_1, d_2, \cdots, d_N) \in \mathbb{R}^{N\times N}$ is degree matrix and d_i is the degree of node i. The different groups of node is overlapping and will be computed multiple times, thus we normalize the coding rate of node representations for groups with the average degree $\bar{d}$ of all nodes. Consequently, the sum of the coding rate of node representations for each group is given as the following:

$$R^c_{\mathcal{G}}(\mathbf{Z}, \epsilon|\mathcal{A}) \doteq \frac{1}{\bar{d}}\sum_{i=1}^N \frac{\text{tr}(\mathbf{A}_i)}{2N} \cdot \log\det\left(\mathbf{I} + \frac{d}{\text{tr}(\mathbf{A}_i)\epsilon^2}\mathbf{Z}\mathbf{A}_i\mathbf{Z}^\top\right), \quad (5)$$

where N is the total number of nodes in the graph, $\bar{d}$ is the average degree of nodes, and $\mathcal{A}$ is the membership matrix set.

3.2 Rate Reduction Maximization for Training

3.2.1 Objective function. Combining Equations (4) and (5), the rate reduction for the graph with adjacency matrix $\mathbf{A}$ is given as follows:

$$\begin{aligned}\Delta R_{\mathcal{G}}(\mathbf{Z}, \mathbf{A}, \epsilon) &= R_{\mathcal{G}}(\mathbf{Z}, \epsilon) - R^c_{\mathcal{G}}(\mathbf{Z}, \epsilon \mid \mathcal{A}) \\ &\doteq \frac{1}{2}\log\det\left(\mathbf{I} + \frac{d}{N\epsilon^2}\mathbf{Z}\mathbf{Z}^\top\right) \\ &\quad - \frac{1}{\bar{d}}\sum_{i=1}^N \frac{\text{tr}(\mathbf{A}_i)}{2N} \cdot \log\det\left(\mathbf{I} + \frac{d}{\text{tr}(\mathbf{A}_i)\epsilon^2}\mathbf{Z}\mathbf{A}_i\mathbf{Z}^\top\right).\end{aligned} \quad (6)$$

In practice, we control the strength of compactness of the node representations by adding two hyperparameters γ_1 and γ_2 to the first term in Equation (6). The γ_1 controls compression of the node representations while the γ_2 balances the coding rate of the entire node representations and that of the groups. Thus we have

$$\begin{aligned}&\Delta R_{\mathcal{G}}(\mathbf{Z}, \mathbf{A}, \epsilon, \gamma_1, \gamma_2) \\ &\doteq \frac{1}{2\gamma_1}\log\det\left(\mathbf{I} + \frac{d\gamma_2}{N\epsilon^2}\mathbf{Z}\mathbf{Z}^\top\right) \\ &\quad - \frac{1}{\bar{d}}\sum_{i=1}^N \frac{\text{tr}(\mathbf{A}_i)}{2N} \cdot \log\det\left(\mathbf{I} + \frac{d}{\text{tr}(\mathbf{A}_i)\epsilon^2}\mathbf{Z}\mathbf{A}_i\mathbf{Z}^\top\right),\end{aligned} \quad (7)$$

where ϵ, γ_1, and γ_2 serve as the hyperparameters of our model.

3.2.2 Model Training. We adopt graph neural network as the encoder to transform the input graph to node representations, where $\mathbf{Z} = \text{GNN}(\mathbf{X}, \mathbf{A}|\theta)$ and θ denotes the parameters to be optimized. The output of the last GNN layer is the learned node representations, which is $L-1$ normalized as mentioned before. The parameters θ will be optimized by maximizing the following objective:

$$\max_\theta \Delta R_{\mathcal{G}}(\text{GNN}(\mathbf{X}, \mathbf{A}|\theta), \mathbf{A}, \epsilon, \gamma_1, \gamma_2), \quad (8)$$

where ϵ, γ_1, and γ_2 serve as the hyperparameters of our model. We also conduct experiments to explore the effect of hyperparameters γ_1 and γ_2 in Section 5.7. We set hyperparameters ϵ to a heuristic value 0.05. For large graphs, the adjacency matrix is large and the length of membership matrix set is $|\mathcal{A}| = N$, thus we need to compute coding rate for groups N times in Equations (5) and (6). To reduce the computational complexity, we randomly sample fixed number N_s rows of adjacency matrix for each training batch. Then we use sampled adjacency matrix to assemble the membership matrix set, which only has N_s membership metrics. Thus we only compute the coding rate N_s times.

3.2.3 Computational Complexity. Due to the commutative property [1] of coding rate, computational complexity of the proposal is not high. In this work, we have $\mathbf{Z} \in \mathbb{R}^{N\times d}$, where d is the dimension of node representations and N is the total number of nodes. So we have $\mathbf{Z}^\top\mathbf{Z} \in \mathbb{R}^{d\times d}$ and $\mathbf{Z}\mathbf{Z}^\top \in \mathbb{R}^{N\times N}$. Even though the computation of $\log\det\left(\mathbf{I} + \frac{d}{m\epsilon^2}\mathbf{Z}\mathbf{Z}^\top\right)$ takes $O(N^3)$ times, we can compute $\log\det\left(\mathbf{I} + \frac{d}{m\epsilon^2}\mathbf{Z}^\top\mathbf{Z}\right)$ instead, which takes $O(d^3)$ times and $d \ll N$. In our experiment setting, we set d to 512. Thus the

[1] Commutative property of coding rate: $R(\mathbf{Z}, \epsilon) \doteq \frac{1}{2}\log\det\left(\mathbf{I} + \frac{d}{m\epsilon^2}\mathbf{Z}\mathbf{Z}^\top\right) = \frac{1}{2}\log\det\left(\mathbf{I} + \frac{d}{m\epsilon^2}\mathbf{Z}^\top\mathbf{Z}\right)$

operation logdet$(\cdot)$ will only take $O(d^3)$ times, which is constant time and does not depend on the nodes number N. Besides, since $\mathbf{Z}\mathbf{Z}^\top \in \mathbb{R}^{d\times d}$, the memory usage will not increase while the number of nodes (N) increases, leading to the scalability of G^{2}R.

3.3 Discussion: what is G^{2}R doing intuitively?

To understand the proposed objective function in Equation (6), we informally discuss the intuition behind it.

- **The first term enforces diverse node representations space.** Maximizing the first term in Equation (6) tends to increase the diversity of representation vectors for all nodes, thus leading to a more diverse distribution of node representations.
- **The second term enforces more similar representations for connected nodes.** The second term in Equation (6) measures the compactness of the representation of node groups. Minimizing the second term enforces the similarity of node representations. As a result, the learned representations of connected nodes will cluster together, as shown in Figure 5.

4 THEORETICAL JUSTIFICATION

To gain a deeper insight of G^{2}R, we theoretically investigate the Equation (6) on an example graph with two communities as a simplified illustration. Consequently, we prove that G^{2}R maps representations of nodes in different communities to different subspaces and aim to maximize the principal angle [2] between different subspaces, thus encouraging them to be (nearly) orthogonal.

4.1 Principal Angle Between Subspaces

To measure the difference between two subspaces, we introduce the *principal angle* θ [23] to generalize the angle between subspaces with arbitrary dimensions. We give the formal definition as follows:

Definition 1 (Principal angle). Given subspace $\mathbf{L}, \mathbf{M} \subseteq \mathbb{R}^n$ with $\dim\mathbf{L} = l \geq \dim\mathbf{M} = m$, there are m principal angles between $\mathbf{L}$ and $\mathbf{M}$ denoted as $0 \leq \theta_1 \leq \theta_2 \leq \cdots \leq \theta_m \leq \frac{\pi}{2}$ between $\mathbf{L}$ and $\mathbf{M}$ are recursively defined, where

$$\cos(\theta_i) := \min\left\{\frac{<\mathbf{x}, \mathbf{y}>}{||\mathbf{x}||||\mathbf{y}||}\middle|\mathbf{x}\in\mathbf{L}, \mathbf{y}\in\mathbf{M}, \mathbf{x}\perp\mathbf{x}_k, \mathbf{y}\perp\mathbf{y}_k, k=1,\cdots,i-1\right\}.$$

We adopt product of sines of principal angles, denoted as $sin\{\mathbf{L},\mathbf{M}\} = sin\theta_1\cdots sin\theta_m \in [0,1]$, to measure the difference between two subspaces. Notably, when two subspaces are orthogonal, the product of principal sines equals 1.

4.2 Graph with Two Communities

Without loss of generality, we analyze the graph with two equal-size communities. We assume each community has M nodes. The graph adjacency matrix $\mathbf{A}$ is generated from the Bernoulli distribution of matrix $\mathbf{P} \in \mathbb{R}^{2M\times 2M}$. The matrix $\mathbf{P}$ is defined as follows:

$$p_{i,j} = \begin{cases} p^i, & \text{if nodes } i,j \text{ are in the same community;} \\ p^o, & \text{otherwise,} \end{cases} \tag{9}$$

where $p_{i,j}$ is the element of matrix $\mathbf{P}$ for i^{th} row and j^{th} column. In other words, the relation between $\mathbf{P}$, $\mathbf{A}$ are shown as follows:

$$\mathbf{P} = \left[\begin{array}{ccc|ccc} p^i & \cdots & p^i & p^o & \cdots & p^o \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ p^i & \cdots & p^i & p^o & \cdots & p^o \\ \hline p^o & \cdots & p^o & p^i & \cdots & p^i \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ p^o & \cdots & p^o & p^i & \cdots & p^i \end{array}\right] \xrightarrow{Bern} \mathbf{A} \in \mathbb{R}^{2M\times 2M}, \tag{10}$$

The i-th row of adjacency matrix $\mathbf{A}$ is denoted as $\mathbf{a}_i = [a_{i1}, \cdots, a_{iN}] \in \mathbb{R}^N$, which is generated from Bernoulli distributions $Bern(\mathbf{P}_{i*})$ independently. To compute the coding rate in graphs, we rewrite the connectivity probability matrix $\mathbf{P}$ as follows:

$$\mathbf{P} = p^o\cdot\mathbb{1}\mathbb{1}^\top + \underbrace{\left[\begin{array}{ccc|ccc} p^i-p^o & \cdots & p^i-p^o & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ p^i-p^o & \cdots & p^i-p^o & 0 & \cdots & 0 \\ \hline 0 & \cdots & 0 & p^i-p^o & \cdots & p^i-p^o \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & p^i-p^o & \cdots & p^i-p^o \end{array}\right]}_{\triangleq \mathbf{C}}, \tag{11}$$

where $\mathbb{1} \in \mathbb{R}^{N\times 1}$ is an all-ones vector and $\mathbb{1}\mathbb{1}^\top \in \mathbb{R}^{N\times N}$ is an all-ones matrix. The first term $p^o\cdot\mathbb{1}\mathbb{1}^\top$ extracts the uniform background factor that is equally applied to all edges. The second term in Equation (11) $\mathbf{C} = [\mathbf{C}_1, \cdots, \mathbf{C}_M, \mathbf{C}_{M+1}, \cdots, \mathbf{C}_{2M}] \in \mathbb{R}^{2M\times 2M}$ tells the difference of node connections in different communities, so we only focus on the second term in the following analysis.

4.3 Coding Rate for Graph with Communities

Since there are two communities, the membership matrices set is defined as $C = \{\mathbf{C}_1, \cdots, \mathbf{C}_M, \mathbf{C}_{M+1}, \cdots, \mathbf{C}_{2M}\}$. Since the $\mathbf{C}_1 = \mathbf{C}_2 = \cdots = \mathbf{C}_M$ and $\mathbf{C}_{M+1} = \mathbf{C}_{M+2} = \cdots = \mathbf{C}_{2M}$, we can rewrite the membership matrix to $C = \{\underbrace{\mathbf{C}^1, \cdots, \mathbf{C}^1}_{M}, \underbrace{\mathbf{C}^2, \cdots, \mathbf{C}^2}_{M}\}$ where $\mathbf{C}^1 = \mathbf{C}_1 = \cdots = \mathbf{C}_M$ and $\mathbf{C}^2 = \mathbf{C}_{M+1} = \cdots = \mathbf{C}_{2M}$.

Thus we soften the Equation (4) by replacing $\mathbf{A}_i$ with its $\mathbf{C}_i$,

$$\begin{aligned} R^c_{\mathcal{G}}(\mathbf{Z}, \epsilon|C) &\doteq \frac{1}{d}\sum_{i=1}^{2M}\frac{\mathrm{tr}(\mathbf{C}_i)}{2N}\cdot\log\det\left(\mathbf{I} + \frac{d}{\mathrm{tr}(\mathbf{C}_i)\epsilon^2}\mathbf{Z}\mathbf{C}_i\mathbf{Z}^\top\right) \\ &\doteq \frac{M}{d}\sum_{i=1}^{2}\frac{\mathrm{tr}(\mathbf{C}^i)}{2N}\cdot\log\det\left(\mathbf{I} + \frac{d}{\mathrm{tr}(\mathbf{C}^i)\epsilon^2}\mathbf{Z}\mathbf{C}^i\mathbf{Z}^\top\right). \end{aligned} \tag{12}$$

The rate reduction will take

$$\begin{aligned} \Delta R_{\mathcal{G}}(\mathbf{Z}, C, \epsilon) &= R_{\mathcal{G}}(\mathbf{Z}, \epsilon) - R^c_{\mathcal{G}}(\mathbf{Z}, \epsilon|C) \\ &= \sum_{j=1}^{2}\log\left(\frac{\det^{\frac{1}{4}}\left(\mathbf{I} + \frac{d}{N\epsilon^2}\mathbf{Z}_j^\top\mathbf{Z}_j\right)}{\det^{\frac{p^i-p^o}{2N}}\left(\mathbf{I} + \frac{d}{M\epsilon^2}\mathbf{Z}_j^\top\mathbf{Z}_j\right)}\right) + \frac{1}{2}\cdot\log\beta. \end{aligned} \tag{13}$$

where $\mathbf{I} + \frac{d}{N\epsilon^2}\mathbf{Z}^\top\mathbf{Z} = \tilde{\mathbf{Z}}^\top\tilde{\mathbf{Z}}$ and $\beta = \sin\{R(\tilde{\mathbf{Z}}_1), R(\tilde{\mathbf{Z}}_2)\}$. The detailed proof is provided in Appendix A.3.

[2] The principal angle measures the difference of subspaces. The higher principal angle indicates more discriminative subspaces.

4.4 Discussion: what is G^2R doing theoretically?

Equation (13) attempts to optimize the principal angle of different subspaces. Different representation subspaces are more distinguishable if β is larger. Thus, maximizing the second term in Equation (13) promises the following desirable properties:

- **Inter-communities.** The representations of nodes in different communities are mutually distinguishable. The node representations of different communities should lie in different subspaces and the principal angle of subspaces are maximized (i.e., nearly pairwise orthogonal), which is verified by experimental results in Figure 2 and Figure 3.
- **Intra-communities.** The representations of nodes in the same community share the same subspace. So the representations of nodes in the same community should be more similar than nodes in different communities.

Based on the above analysis, G^2R achieves *geometric* representation learning by constraining the distribution of node representations in different subspaces and encouraging different subspaces to be orthogonal. The *geometric* information considers a broader scope of embedding distribution in latent space.

5 EXPERIMENTS

In this section, we conduct experiments with synthetic graph and real-world graphs to comprehensively evaluate G^2R. The main observations in experiments are highlighted as **(#) boldface**.

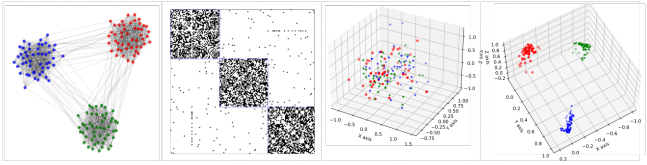
5.1 What is G^2R Doing? Empirical Verification with Synthetic Graph Data

We experiment with a synthetic graph to empirically verify that G^2R tends to project node representations in different communities into different subspaces. The results are presented in Figure 2.

5.1.1 Synthetic Graph Generation. The synthetic graph is generated as follows: i) Graph structure. We partition nodes into 3 balanced communities and construct edges with Gaussian random partition[3]. The nodes within the same community have a high probability 0.5 to form edges and a lower probability 0.01 for nodes in different communities. Figure 2(a) and Figure 2(b) show the structure of the synthetic graph and its adjacency matrix, respectively. ii) Node features. The node feature is generated from multivariate Gaussian distributions with the same mean and standard deviation, the dimension of which is 32. t-SNE [36] of node features to 3-dimensional space are in Figure 2(c). Figure 2(d) is the visualization of the learned node representations, the dimension of which is 3.

5.1.2 Results. Comparing Figures 2(c) and 2(d), we observed that **(1) the learned node representations in different communities are nearly orthogonal** in the three-dimensional space. Moreover, we also compute the cosine similarity between each pair of the node representations to quantify the geometric relation and we observe that the cosine similarity scores for node representations pair between the different communities are extremely close to 0. This observation indicates that G^2R tends to maximize the principal angle of representation spaces of nodes in different communities. **(2)**

[3]https://networkx.org



(a) Synthetic graph (b) Adjacency matric (c) Original node features (d) Learned node representation

Figure 2: Synthetic graph and visualization of its node features and representations. The different colors in (a)(c)(d) indicate different communities. The learned node representations in (d) are 3-dimensional vectors obtained by G^2R.

The node representations in the same community are compact. Figure 2(c) shows the original features of nodes in the same color are loose while node representations in Figure 2(d) in the same color cluster together. This observation shows that G^2R can compact the node representations in the same community. The experimental results on synthetic data are remarkably consistent with the theoretical analysis in Section 4.4 that the node representations in different communities will be (nearly) orthogonal.

5.2 Will G^2R Perform Better than Unsupervised Counterparts?

We contrast the performance of the node classification task of G^2R and various unsupervised baselines.

5.2.1 Experiment Setting. For dataset, we experiment on eight real-world datasets, including citation network [1, 42] (Cora, CiteSeer, PubMed, CoraFull), co-authorship networks [33] (Physics, CS), and Amazon co-purchase networks [22] (Photo, Computers). The details of datasets are provided in Appendix B.3. *For baselines*, we compare three categories of unsupervised baselines. The first category only utilizes node features, including original node features, PCA [39], SVD [12], LLE [32] and Isomap [35]. The second only considers adjacency information, including DeepWalk [30] and Node2vec [13]. The third considers both, including DGI [38], GMI [29], GRACE [44] and GraphCL [44]. *For evaluation*, we follow the linear evaluation scheme adopted by [38, 48], which first trains models in an unsupervised fashion and then output the node representations to be evaluated by a logistic regression classifier [2]. We use the same random train/validation/test split as [8, 20]. To ensure a fair comparison, we use 1)the same logistic regression classifier, and 2)the same data split for all models. The results are summarized in Table 1.

5.2.2 Results. From Table 1, we observe that **(3)** G^2R **outperforms all baselines by significant margins on seven datasets among eight dataset.** Except for the Photo dataset, G^2R achieves the state-of-the-art performance by significant margins. The average percentage of improvement to DGI (representative unsupervised method) and GRACE (representative contrastive learning method) is 5.1% and 5.9%, respectively. Moreover, G^2R is capable of handling large graph data. The reason is partial leverage of adjacency matrix in each training batch requires lower memory usage and less time.

Table 1: Performance comparison to unsupervised methods. The accuracy with standard deviation are based on 5 runs for all methods. The second column shows the information used by the method, where X, A denote node features and adjacency matrix, respectively. 'OOM' means out of memory while running on NVIDIA RTX3090 (24GB memory). 'Public/Random' represents the public/random data split. The best performance among baselines is underlined. The best performance is in boldface.

Statistic		Cora		CiteSeer		PubMed		CoraFull	CS	Physics	Computers	Photo
#Nodes		2708		3327		19717		19793	18333	34493	13381	7487
#Edges		5278		4552		44324		130622	81894	24762	245778	119043
#Features		1433		3703		500		8710	6805	8415	767	745
#Density		0.0014		0.0008		0.0002		0.0003	0.0005	0.0004	0.0027	0.0042
#Classes		7		6		3		70	15	5	10	8
#Data Split		140/500/1000		120/500/1000		60/500/1000		1400/2100/rest	300/450/rest	100/150/rest	200/300/rest	160/240/rest
Metric	**Feature**	**Public**	**Random**	**Public**	**Random**	**Public**	**Random**	**Random**	**Random**	**Random**	**Random**	**Random**
Feature	X	58.90±1.35	60.19±0.00	58.69±1.28	61.70±0.00	69.96±2.89	73.90±0.00	40.06±1.07	88.14±0.26	87.49±1.16	67.48±1.48	59.52±3.60
PCA	X	57.91±1.36	59.90±0.00	58.31±1.46	60.00±0.00	69.74±2.79	74.00±0.00	38.46±1.13	88.59±0.29	87.66±1.05	72.65±1.43	57.45±4.38
SVD	X	58.57±1.30	60.21±0.19	58.10±1.14	60.80±0.26	69.89±2.66	73.79±0.29	38.64±1.11	88.55±0.31	87.98±1.10	68.17±1.39	60.98±3.58
isomap	X	40.19±1.24	44.60±0.00	18.20±2.49	18.90±0.00	62.41±3.65	63.90±0.00	4.21±0.25	73.68±1.25	82.84±0.81	72.66±1.38	44.00±6.43
LLE	X	29.34±1.24	36.70±0.00	18.26±1.60	21.80±0.00	52.82±2.08	54.00±0.00	5.70±0.38	72.23±1.57	81.35±1.59	45.29±1.31	35.37±1.82
DeepWalk	A	74.03±1.99	73.76±0.26	48.04±2.59	51.80±0.62	68.72±1.43	71.28±1.07	51.65±0.83	83.25±0.54	88.08±1.45	86.47±1.55	76.58±1.09
Node2vec	A	73.64±1.94	72.54±1.12	46.95±1.24	49.37±1.53	70.17±1.39	68.70±0.96	50.35±0.74	82.12±1.09	86.77±0.83	85.15±1.32	75.67±1.98
DeepWalk+F	X, A	77.36±0.97	77.62±0.27	64.30±1.01	66.96±0.30	69.65±1.84	71.84±1.15	54.63±0.74	83.34±0.53	88.15±1.45	**86.49**±1.55	65.97±3.68
Node2vec+F	X, A	75.44±1.80	76.84±0.25	63.22±1.50	66.75±0.74	70.6±1.36	69.12±0.96	54.00±0.17	82.20±1.09	86.86±0.80	85.15±1.33	65.01±2.91
GAE	X, A	73.68±1.08	74.30±1.42	58.21±1.26	59.69±3.29	76.16±1.81	80.08±0.70	42.54±2.69	88.88±0.83	91.01±0.84	37.72±9.01	48.72±5.28
VGAE	X, A	77.44±2.20	76.42±1.26	59.53±1.06	60.37±1.40	78.00±1.94	77.75±0.77	53.69±1.32	88.66±1.04	90.33±1.77	49.09±5.95	48.33±1.74
DGI	X, A	81.26±1.24	82.11±0.25	69.50±1.29	70.15±1.10	77.70±3.17	79.06±0.51	53.89±1.38	91.22±0.48	92.12±1.29	79.62±3.31	70.65±1.72
GRACE	X, A	80.46±0.05	80.36±0.51	68.72±0.04	68.04±1.06	80.67±0.04	OOM	53.95±0.11	90.04±0.11	OOM	81.94±0.48	70.38±0.46
GraphCL	X, A	81.89±1.34	81.12±0.04	68.40±1.07	69.67±0.13	OOM	81.41±0.10	OOM	OOM	OOM	79.90±2.05	OOM
GMI	X, A	80.28±1.06	81.20±0.78	65.99±2.75	70.50±0.36	OOM	OOM	OOM	OOM	OOM	52.36±5.22	OOM
G^2R**(ours)**	X, A	**82.58**±1.41	**83.32**±0.75	**71.2**±1.01	**70.66**±0.49	**81.69**±0.98	**81.69**±0.42	**59.70**±0.59	**92.64**±0.40	**94.93**±0.07	82.24±0.71	**90.68**±0.31

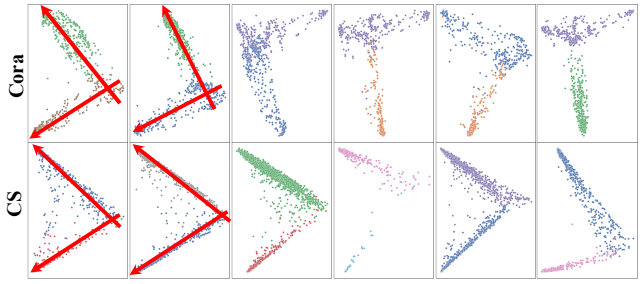


Figure 3: PCA[4] visualization of node representations learned by G^2R on Cora and CS dataset. Every figure only has two classes of nodes and they are nearly orthogonal. Different colors represent different classes. The direction of → in the first two columns show the (nearly) orthogonality of node representations in the two classes.

5.3 Will Representation learned by G^2R (nearly) orthogonal? Visualization Analysis

We perform a visualization experiment to analyze the representations learned by G^2R to verify its effectiveness further. The visualization of nodes representations of different classes is in Figure 3.

5.3.1 Results. Figure 3 remarkably shows that **(4) the representations of nodes in different classes learned by G^2R are nearly orthogonal to each other.** Since the nodes in the same class typically connected densely, leading them to be nearly orthogonal to each other according to the proof in Section 4. This observation also strongly supports our theoretical analysis Section 4.

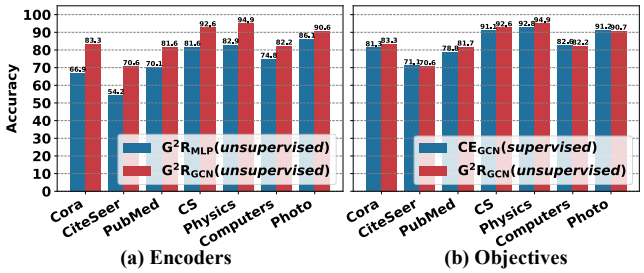


Figure 4: Ablation studies. (a): We compare the performance of different encoders by instantiating the encoder as GCN or MLP, respectively. (b): we compare the performance of different objective functions, Cross-Entropy(CE, supervised method for node classification) and ours (unsupervised).

5.4 What is the Effect of Encoders and Objective Functions? Ablation Studies

We investigate the effect of encoder and objective function in G^2R using ablation studies. Specifically, we replace the graph neural networks in G^2R with other encoders or replace the proposed objective functions with cross-entropy. The results are in Figure 4.

5.4.1 Results. Figures 4(a) and 4(b) show that **(5) the superiority of G^2R_{GCN} is attributed to the graph neural network and the proposed objective.** Figure 4(a) indicates that graph neural networks as the encoder significantly improve the effectiveness of G^2R. Figure 4(b) shows that performance of CE_{GCN} drops significantly

[4]The reason why we choice PCA here is that PCA will preserve the orthogonality between vectors when transform the high-dimensional vectors to low-dimension [16]. Each figure includes two classes of node since we display node representation in two-dimensional space.

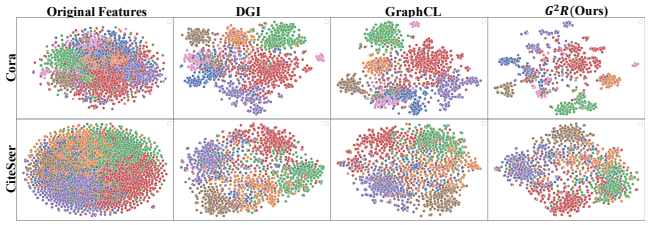


Figure 5: t-SNE [36] visualization of original node features and node representations learned by different methods. Different colors represent different classes.

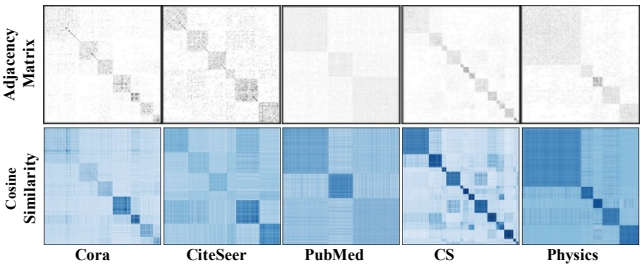


Figure 6: Visualization of adjacency matrices and the cosine similarity of learned node representations. Notably, cosine similarity can visually "recover" the adjacency matrix.

compared to the G^2R_{GCN} even though the it is a supervised method for node classification. This observation indicates that superoity of G^2R largely stems from the proposed objective function.

5.5 Will the Graph Structure be Preserved in the Learned Representation?

To investigate whether the learned node representations preserves the graph structure, we perform two visualization experiments, including **1)** t-SNE [36] visualization of the original features and the node representations learned by different methods in Figure 5, and **2)** visualization of the adjacency metrics of graphs and cosine similarity between learned node representations $\mathbf{Z}$ in Figure 6.

5.5.1 Results. From Figure 5, **(6) the distinctly separable clusters demonstrate the discriminative capability of** G^2R. The node representations learned by G^2R are more compact within class, leading to the discriminative node representations. The reason is that G^2R can map the nodes in different communities into different subspaces and maximize the difference of these subspaces. Figure 6 shows that **(7)** G^2R **is able to map the nodes representations in the different communities to different subspace and thus implicitly preserve the graph structure.** The cosine similarity of the node representations can noticeably "recover" the adjacency matrix of the graph, demonstrating the learned node representations preserved the graph structure.

5.6 Will Learned Representation Perform Well on Community Detection? A Case Study

We conduct community detection on the Cora dataset using the learned node representations by G^2R.

5.6.1 Experimental Setting. We conduct community detection by applying K-Means to node representations learned by G^2R and use

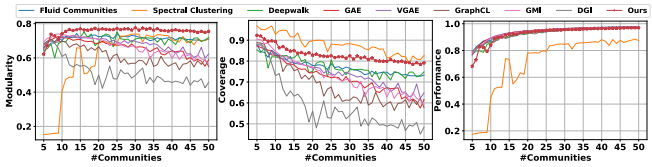


Figure 7: Performance of community detection on Cora dataset. The x-axis indicates the number of communities to be detected, the y-axis indicates the different metrics of community detection,including modularity, performance and coverage (higher is better for all metrics).

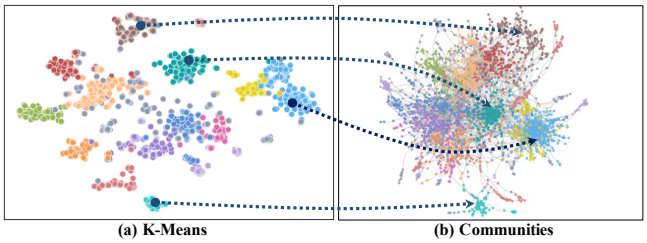


Figure 8: Community detection on Cora. We detect the communities only utilizing the learned node representations. Specifically, we use K-Means to cluster the node representations and the nodes with the same cluster label is in identical community. According to the communities label obtained from the node representations, we draw the original graph and color the communities in the same colors to Figure 8. The same color indicates the same community.

the predicted cluster labels as communities. We use traditional community detection methods as baselines, including asynchronous fluid communities algorithm [27] and spectral clustering [26]. We also use the node representations learned by other unsupervised methods as baselines. The metrics to evaluate the community detection are modularity [4], coverage, performance.[5] The results are in Figure 7. We also show a case of community detection in Figure 8.

5.6.2 Results. Figures 7 and 8, quantitatively and qualitatively, show **(8)** G^2R **outperforms the traditional community detection methods as well as unsupervised baselines for community detection task.** Figure 7 shows that G^2R outperforms various community detection methods by a large margin on three metrics. In Figure 8, communities detected in Cora are visually consistent with the node representations clusters. The better performance of G^2R results from the orthogonality of different subspaces, into which the nodes in different communities are projected.

5.7 What is the Effect of the Hyperparameters γ_1nd γ_2?

We investigate the effect of hyperparameters γ_1 and γ_2 on G^2R via training with 20 evenly spaced values of both γ_1 and γ_2 within $(0, 1]$ on Cora, CiteSeer, PubMed datasets. The results are presented in Figure 9. From Figure 9, we observed that **(9) hyperparameters strongly influence the performance of** G^2R **and the best performance is achieved around** $\gamma_1 = \gamma_2 = 0.5$ The performance is lower while $\gamma_1 < 0.5$ and $\gamma_2 < 0.5$, which shows that it is important to control the dynamics of the expansion and compression of

[5]The detail about these metric are presented in Appendix C

the node representations. **(10)** G^2R **is not sensitive to hyperparameter across different datasets**, since G^2R achieves the best performance with the similar hyperparameters ($\gamma_1 = \gamma_2 = 0.5$) on Cora, CiteSeer, PubMed datasets. Based on this observation, we set $\gamma_1 = \gamma_2 = 0.5$ on all datasets in our performance experiments.

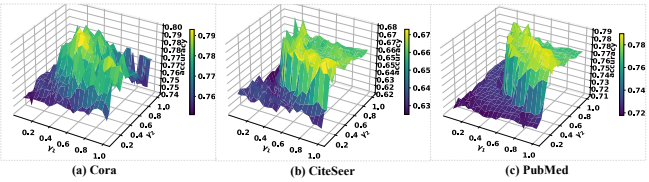


Figure 9: The effect of the hyperparameters γ_1 and γ_2 on node classification task. The x-axis indicates the values of γ_1, the y-axis indicates the values of γ_2 and the z-axis indicates the model performance on node classification task.

5.8 G^2R is even Better than Supervised Counterparts

Despite that G^2R shows its superior performance compared to the unsupervised baselines, we contrast the performance of G^2R and supervised methods on the node classification task.

5.8.1 Experiments Settings. We consider the following supervised learning baselines: Logistic Regression (LogReg), Multilayer Perceptron (MLP), Label Propagation (LP) [3], Normalized Laplacian Label Propagation (LP NL) [3], Cheb-Net [7], Graph Convolutional Network (GCN) [18], Graph Attention Network (GAT) [37], Mixture Model Network (MoNet) [25], GraphSAGE(SAGE) [14], APPNP [19], SGC [40] and DAGNN [20]. The results of the baselines are obtained from [20, 33], so we follow the same data split and the same datasets in the papers [20, 33]. We follow the linear evaluation scheme for G^2R, where G^2R was trained in an unsupervised manner and then output the node representations as input features to a logistic regression classifier [2]. The details of baselines are provided in Appendix B.4. The results are summarized in Table 2.

5.8.2 Results. From Table 2, we observed that **(11)** G^2R **shows comparable performance across all seven datasets, although the baselines are all supervised methods.** From the 'Avg.rank' column in Table 2, G^2R ranks 3.1 among all the methods on all datasets. G^2R obtains a comparable performance in node classification task even though compared to supervised baselines. This observation shows the node representations learned by G^2R preserve the information for node classification task even though compared to the end-to-end models for the same downstream task.

Table 2: Comparison to supervised baselines with public data split on node classification. 'P' means public data split while 'R' means random data split. 'Phy./Com./Pho.' means Physics/Computers/Photo dataset. The 'Avg.Rank' is the average rank among all the methods on all datasets.

Methods	Cora		CiteSeer		PubMed		CS	Phy.	Com.	Pho.	Avg.
	P	R	P	R	P	R					Rank
LogReg	52.0	58.3	55.8	60.8	73.6	69.7	86.4	86.7	64.1	73.0	**11.3**
MLP	61.6	59.8	61.0	58.8	74.2	70.1	88.3	88.9	44.9	69.6	**10.9**
LP	71.0	79.0	50.8	65.8	70.6	73.3	73.6	86.6	70.8	72.6	**11.2**
LP NL	71.2	79.7	51.2	66.9	72.6	77.8	76.7	86.8	75.0	83.9	**9.5**
ChebNet	80.5	76.8	69.6	67.5	78.1	75.3	89.1	-	15.2	25.2	**10.0**
GCN	81.3	79.1	71.1	68.2	78.8	77.1	91.1	92.8	82.6	91.2	**5.7**
GAT	83.1	80.8	70.8	68.9	79.1	77.8	90.5	92.5	78.0	85.7	**5.8**
MoNet	79.0	84.4	70.3	71.4	78.9	83.3	90.8	92.5	83.5	91.2	**4.0**
SAGE	78.0	84.0	70.1	71.1	78.8	79.2	91.3	93.0	82.4	91.4	**4.7**
APPNP	83.3	81.9	71.8	69.8	80.1	79.5	90.1	90.9	20.6	30.0	**6.0**
SGC	81.7	80.4	71.3	68.7	78.9	76.8	90.8	-	79.9	90.7	**5.9**
DAGNN	84.4	83.7	73.3	71.2	80.5	80.1	92.8	94.0	84.5	92.0	**1.7**
Ours	83.3	82.6	70.6	71.2	81.7	81.7	92.6	94.9	82.2	90.7	**3.1**

6 RELATED WORKS

Graph representation learning with random walks. Many approaches [13, 30, 31, 34] learn the node representations based on random walk sequences. Their key innovation is optimizing the node representations so that nodes have similar representations if they tend to co-occur over the graph. In our experiment, we use DeepWalk and node2vec as baselines, which are the representative methods based on random walk. DeepWalk [30], as pioneer work to learn representations of vertices in a network, uses local information from truncated random walks as input to learn a representation which encodes structural regularities. node2vec [13] aims to map nodes into a low-dimensional space while maximizing the likelihood of preserving nodes neighborhoods.

Contrastive graph representation learning. Contrastive learning is the key component to word embedding methods [5, 24], and recently it is used to learn representations for graph-structured data [10, 13, 14, 18, 30]. For example, DGI [38] learns node representations in an unsupervised manner by maximizing mutual information between patch representations and the graph representation. GRACE [44] maximizes the agreement of node representations in two generated views. GraphCL [44] learns representations with graph data augmentations.

Graph Neural Networks. Graph neural networks have became the new state-of-the-art approach to process graph data [14, 15]. Starting with the success of GCN in the semi-supervised node classification task[18], a wide variety of GNN variants have proposed for graph learning task [9, 14, 37, 38, 40]. Most of them follow a message passing strategy to learn node representations over a graph. Graph Attention Network (GAT) [37] proposes masked self-attentional layers that allow weighing nodes in the neighborhood differently during the aggregation step. GraphSAGE [14] focuses on inductive node classification with different neighbor sampling strategies. Simple Graph Convolution (SGC) [40] reduces the excess complexity of GCNs by removing the nonlinearities between GCN layers and collapsing the resulting function into a single linear transformation. Personalized propagation of neural predictions (PPNP) and (APPNP) [19] leverage adjustable neighborhood for classification and can be easily combined with any neural network. However, all these methods are typically supervised, which highly rely on reliable labels. In this work, we leverage the graph neural network to encode the graph to node representations.

7 CONCLUSION

Graph representation learning becomes a dominant technique in analyzing graph-structured data. In this work, we propose Geometric Graph Representation Learning (G^2R), an unsupervised approach to learning discriminative node representations for graphs. Specifically, we propose an objective function to enforce discriminative node representations via maximizing the principal angle of the

subspace of different node groups. And we provide theoretical justification for the proposed objective function, which can guarantee the orthogonality for node in different groups. We demonstrate competitive performance of G^2R on node classification and community detection tasks. Moreover, G^2R even outperforms multiple supervised counterparts on node classification task. The strength of G^2R suggests that, despite a recent surge in deeper graph neural networks, unsupervised learning on graph remains promising.

ACKNOWLEDGMENTS

We would like to thank all the anonymous reviewers for their valuable suggestions. This work is in part supported by NSF IIS-1849085, CNS-1816497, IIS-1750074, and IIS-2006844. The views and conclusions contained in this paper are those of the authors and should not be interpreted as representing any funding agencies.

REFERENCES

[1] Aleksandar Bojchevski and Stephan Günnemann. 2018. Deep Gaussian Embedding of Graphs: Unsupervised Inductive Learning via Ranking. In *ICLR*.
[2] Lars Buitinck, Gilles Louppe, Mathieu Blondel, Fabian Pedregosa, Andreas Mueller, Olivier Grisel, Vlad Niculae, Peter Prettenhofer, Alexandre Gramfort, Jaques Grobler, Robert Layton, Jake VanderPlas, Arnaud Joly, Brian Holt, and Gaël Varoquaux. 2013. API design for machine learning software: experiences from the scikit-learn project. In *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*. 108–122.
[3] Olivier Chapelle, Bernhard Scholkopf, and Alexander Zien. 2009. Semi-supervised learning (chapelle, o. et al., eds.; 2006)[book reviews]. *IEEE Transactions on Neural Networks* 20, 3 (2009), 542–542.
[4] Aaron Clauset, Mark EJ Newman, and Cristopher Moore. 2004. Finding community structure in very large networks. *Physical review E* 70, 6 (2004), 066111.
[5] Ronan Collobert and Jason Weston. 2008. A unified architecture for natural language processing: Deep neural networks with multitask learning. In *Proceedings of the 25th ICML*. 160–167.
[6] Nicola De Cao and Thomas Kipf. 2018. MolGAN: An implicit generative model for small molecular graphs. *arXiv preprint arXiv:1805.11973* (2018).
[7] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional neural networks on graphs with fast localized spectral filtering. In *NeurIPS*. 3844–3852.
[8] Matthias Fey and Jan E. Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
[9] Hongyang Gao and Shuiwang Ji. 2019. Graph u-nets. In *ICML*. PMLR, 2083–2092.
[10] Alberto García-Durán and Mathias Niepert. 2017. Learning graph representations with embedding propagation. *arXiv preprint arXiv:1710.03059* (2017).
[11] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. 249–256.
[12] Gene H Golub and Christian Reinsch. 1971. Singular value decomposition and least squares solutions. In *Linear algebra*. Springer, 134–151.
[13] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 855–864.
[14] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *NeurIPS*. 1024–1034.
[15] Xiao Huang, Qingquan Song, Yuening Li, and Xia Hu. 2019. Graph recurrent networks with attributed random walks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 732–740.
[16] Ian T Jolliffe. 1995. Rotation of principal components: choice of normalization constraints. *Journal of Applied Statistics* 22, 1 (1995), 29–35.
[17] Diederik P Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *ICLR*.
[18] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
[19] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Predict then Propagate: Graph Neural Networks meet Personalized PageRank. In *ICLR*.
[20] Meng Liu, Hongyang Gao, and Shuiwang Ji. 2020. Towards deeper graph neural networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 338–348.
[21] Yi Ma, Harm Derksen, Wei Hong, and John Wright. 2007. Segmentation of multivariate mixed data via lossy data coding and compression. *IEEE transactions on pattern analysis and machine intelligence* 29, 9 (2007), 1546–1562.
[22] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton Van Den Hengel. 2015. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 43–52.
[23] Jianming Miao and Adi Ben-Israel. 1992. On principal angles between subspaces in Rn. *Linear algebra and its applications* 171 (1992), 81–98.
[24] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Distributed representations of words and phrases and their compositionality. *arXiv preprint arXiv:1310.4546* (2013).
[25] Federico Monti, Davide Boscaini, Jonathan Masci, Emanuele Rodola, Jan Svoboda, and Michael M Bronstein. 2017. Geometric deep learning on graphs and manifolds using mixture model cnns. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 5115–5124.
[26] Andrew Y Ng, Michael I Jordan, and Yair Weiss. 2002. On spectral clustering: Analysis and an algorithm. In *NeurIPS*. 849–856.
[27] Ferran Parés, Dario Garcia Gasulla, Armand Vilalta, Jonatan Moreno, Eduard Ayguadé, Jesús Labarta, Ulises Cortés, and Toyotaro Suzumura. 2017. Fluid communities: A competitive, scalable and diverse community detection algorithm. In *International Conference on Complex Networks and their Applications*. Springer, 229–240.
[28] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library.. In *NeurIPS*.
[29] Zhen Peng, Wenbing Huang, Minnan Luo, Qinghua Zheng, Yu Rong, Tingyang Xu, and Junzhou Huang. 2020. Graph representation learning via graphical mutual information maximization. In *Proceedings of The Web Conference 2020*. 259–270.
[30] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 701–710.
[31] Jiezhong Qiu, Yuxiao Dong, Hao Ma, Jian Li, Kuansan Wang, and Jie Tang. 2018. Network embedding as matrix factorization: Unifying deepwalk, line, pte, and node2vec. In *Proceedings of the eleventh ACM international conference on web search and data mining*. 459–467.
[32] Sam T Roweis and Lawrence K Saul. 2000. Nonlinear dimensionality reduction by locally linear embedding. *science* 290, 5500 (2000), 2323–2326.
[33] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868* (2018).
[34] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, and Qiaozhu Mei. 2015. Line: Large-scale information network embedding. In *Proceedings of the 24th international conference on world wide web*. 1067–1077.
[35] Joshua B Tenenbaum, Vin De Silva, and John C Langford. 2000. A global geometric framework for nonlinear dimensionality reduction. *science* 290, 5500 (2000), 2319–2323.
[36] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, 11 (2008).
[37] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *ICLR*.
[38] Petar Veličković, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. 2018. Deep Graph Infomax. In *ICLR*.
[39] Svante Wold, Kim Esbensen, and Paul Geladi. 1987. Principal component analysis. *Chemometrics and intelligent laboratory systems* 2, 1-3 (1987), 37–52.
[40] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying graph convolutional networks. In *ICML*. PMLR, 6861–6871.
[41] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. 2020. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* (2020).
[42] Zhilin Yang, William W Cohen, and Ruslan Salakhutdinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *Proceedings of the 33rd ICML*.
[43] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 974–983.
[44] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph contrastive learning with augmentations. *NeurIPS* (2020).
[45] Yaodong Yu, Kwan Ho Ryan Chan, Chong You, Chaobing Song, and Yi Ma. 2020. Learning diverse and discriminative representations via the principle of maximal coding rate reduction. *NeurIPS* 33 (2020).
[46] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2020. Graph neural networks: A review of methods and applications. *AI Open* 1 (2020), 57–81.
[47] X Zhu and Z Ghahramani. 2002. Learning from labeled and unlabeled data with label propagation. *Center for Automated Learning and Discovery, CMU: Carnegie*

Mellon University, USA. (2002).
[48] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. 2020. Deep graph contrastive representation learning. *arXiv preprint arXiv:2006.04131* (2020).
[49] Marinka Zitnik and Jure Leskovec. 2017. Predicting multicellular function through multi-layer tissue networks. *Bioinformatics* 33, 14 (2017), i190–i198.

A THEORETICAL ANALYSIS

A.1 Preliminaries

Theorem 1. [23] Let $\mathbf{A} = (\mathbf{A}_1, \mathbf{A}_2)$, $\mathbf{A}_1 \in \mathbb{R}_l^{n\times n_1}$, $\mathbf{A}_1 \in \mathbb{R}_m^{n\times n_2}$, and $\text{rank}(\mathbf{A}) = l + m$. Then

$$\text{vol}(\mathbf{A}) = \text{vol}(\mathbf{A}_1)\text{vol}(\mathbf{A}_2)\sin\{R(\mathbf{A}_1), R(\mathbf{A}_2)\},$$

where $\text{vol}(\mathbf{A}) = \sqrt{\det(\mathbf{A}^\top\mathbf{A})}$ measures the compactness of $\mathbf{A}$ and $\sin\{R(\mathbf{A}_1), R(\mathbf{A}_2)\}$ is the product of principal sines between $R(\mathbf{A}_1)$ and $R(\mathbf{A}_2)$.

Corollary 1. Let $\mathbf{I} + a\mathbf{Z}^\top\mathbf{Z}$ and $\mathbf{Z} = [\mathbf{z}_1, \mathbf{z}_2, \cdots, \mathbf{z}_n] \in \mathbb{R}^{d\times n}$, while the $\mathbf{z}_i$ in $\mathbf{Z}$ are pairwise orthogonal, then the $\tilde{\mathbf{z}}_i$ in $\tilde{\mathbf{Z}}$ are pairwise orthogonal.

Proof: Suppose $\mathbf{Z} = \mathbf{U}\Sigma\mathbf{V}^\top$, then we have

$$\mathbf{I} + a\mathbf{Z}^\top\mathbf{Z} = \mathbf{I} + a\mathbf{U}\Sigma^2\mathbf{V}^\top = \mathbf{U}(I + a\Sigma^2)\mathbf{V}^\top = \tilde{\mathbf{Z}}^\top\tilde{\mathbf{Z}}. \tag{14}$$

We can see from the above derivation, while the $\mathbf{z}_n$ in $\mathbf{Z}$ are pairwise orthogonal, the result of $\mathbf{Z}^\top\mathbf{Z}$ is a diagonal matrix, then $\mathbf{U}\Sigma^2\mathbf{V}^\top$ is diagonal matrix, thus $\tilde{\mathbf{Z}}^\top\tilde{\mathbf{Z}} = \mathbf{U}(I + a\Sigma^2)\mathbf{V}^\top$ is diagonal matrix. So the $\tilde{\mathbf{z}}_n$ in $\tilde{\mathbf{Z}}$ are pairwise orthogonal.

A.2 Insights of Coding Rate.

We first present how to derive the coding rate of entire node representations following [21].

Suppose we have data $\mathbf{W} = (w_1, w_2, \cdots, w_m)$, and let ϵ^2 be the error allowable for encoding every vector w_i in $\mathbf{W}$. In other words, we are allowed to distort each vector of w_i with random variable z_i of variance ϵ^2/n. So we have

$$\hat{w}_i = w_i + z_i, \text{with } z_i = \mathcal{N}(0, \frac{\epsilon^2}{n}\mathbf{I}), \tag{15}$$

Then the covariance matrix of w_i is

$$\hat{\Sigma} \doteq \mathbb{E}[\frac{1}{m}\sum_{i=1}^{m}\hat{w}_i\hat{w}_i^\top] = \frac{\epsilon^2}{n}\mathbf{I} + \frac{1}{m}\mathbf{W}\mathbf{W}^\top, \tag{16}$$

And the volumes of covariance matrix and random vector z_i are

$$\begin{aligned} \text{vol}(\hat{\mathbf{W}}) &\propto \sqrt{\det(\frac{\epsilon^2}{n}\mathbf{I} + \frac{1}{m}\mathbf{W}\mathbf{W}^\top)}, \\ \text{vol}(z) &\propto \sqrt{\det(\frac{\epsilon^2}{n}\mathbf{I})}, \end{aligned} \tag{17}$$

Then the number of bit needed to encode the data $\mathbf{W}$ is

$$R(\mathbf{W}) = \log_2(\frac{\text{vol}(\hat{\mathbf{W}})}{\text{vol}(z)}) = \frac{1}{2}\log_2\det(\mathbf{I} + \frac{n}{m\epsilon^2}\mathbf{W}\mathbf{W}^\top). \tag{18}$$

A.3 Proof of Equation (13)

We take $\mathbf{I} + \frac{d}{N\epsilon^2}\mathbf{Z}^\top\mathbf{Z} = \tilde{\mathbf{Z}}^\top\tilde{\mathbf{Z}}$ and $\beta = \sin\{R(\tilde{\mathbf{Z}}_1), R(\tilde{\mathbf{Z}}_2)\}$, then we have

$$\begin{aligned}
&\Delta R_{\mathcal{G}}(\mathbf{Z}, C, \epsilon) \\
&= R_{\mathcal{G}}(\mathbf{Z}, \epsilon) - R_{\mathcal{G}}^c(\mathbf{Z}, \epsilon|C) \\
&= \frac{1}{2}\text{logdet}\left(\mathbf{I} + \frac{d}{N\epsilon^2}\mathbf{Z}^\top\mathbf{Z}\right) - \sum_{j=1}^{2M}\frac{tr(\mathbf{C}_j)}{2N}\text{logdet}\left(\mathbf{I} + \frac{d}{tr(\mathbf{C}_j)\epsilon^2}\mathbf{Z}^\top\mathbf{C}_j\mathbf{Z}\right) \\
&= \frac{1}{2}\text{logdet}\left(\mathbf{I} + \frac{d}{N\epsilon^2}\mathbf{Z}^\top\mathbf{Z}\right) - \frac{1}{M}\sum_{j=1}^{2}\frac{tr(\mathbf{C}^j)}{2N}\text{logdet}\left(\mathbf{I} + \frac{d}{tr(\mathbf{C}^j)\epsilon^2}\mathbf{Z}^\top\mathbf{C}^j\mathbf{Z}\right) \\
&= \frac{1}{2}\text{logdet}\left(\mathbf{I} + \frac{d}{N\epsilon^2}\mathbf{Z}^\top\mathbf{Z}\right) - \frac{1}{M}\sum_{j=1}^{2}\frac{(p^i - p^o)M}{2N}\text{logdet}\left(\mathbf{I} + \frac{d\cdot(p^i - p^o)}{M\cdot(p^i - p^o)\cdot\epsilon^2}\mathbf{Z}_j^\top\mathbf{Z}_j\right) \\
&= \frac{1}{2}\text{logdet}\left(\mathbf{I} + \frac{d}{N\epsilon^2}\mathbf{Z}^\top\mathbf{Z}\right) - \sum_{j=1}^{2}\frac{(p^i - p^o)}{2N}\text{logdet}\left(\mathbf{I} + \frac{d}{M\epsilon^2}\mathbf{Z}_j^\top\mathbf{Z}_j\right) \\
&= \frac{1}{2}\text{logdet}\left(\tilde{\mathbf{Z}}^\top\tilde{\mathbf{Z}}\right) - \sum_{j=1}^{2}\frac{(p^i - p^o)}{2N}\text{logdet}\left(\mathbf{I} + \frac{d}{M\epsilon^2}\mathbf{Z}_j^\top\mathbf{Z}_j\right) \\
&= \frac{1}{2}\sum_{j=1}^{2}\frac{1}{2}\text{logdet}\left(\tilde{\mathbf{Z}}_j^\top\tilde{\mathbf{Z}}_j\right) + \frac{1}{2}\cdot\log\beta - \sum_{j=1}^{2M}\frac{(p^i - p^o)}{2}\text{logdet}\left(\mathbf{I} + \frac{d}{M\epsilon^2}\mathbf{Z}_j^\top\mathbf{Z}_j\right) \\
&= \sum_{j=1}^{2}\frac{1}{4}\text{logdet}\left(\mathbf{I} + \frac{d}{N\epsilon^2}\mathbf{Z}_j^\top\mathbf{Z}_j\right) - \sum_{j=1}^{2}\frac{(p^i - p^o)}{2N}\cdot\text{logdet}\left(\mathbf{I} + \frac{d}{M\epsilon^2}\mathbf{Z}_j^\top\mathbf{Z}_j\right) + \frac{1}{2}\cdot\log\beta \\
&= \sum_{j=1}^{2}\log\left(\frac{\det^{\frac{1}{4}}\left(\mathbf{I} + \frac{d}{N\epsilon^2}\mathbf{Z}_j^\top\mathbf{Z}_j\right)}{\det^{\frac{p^i - p^o}{2N}}\left(\mathbf{I} + \frac{d}{M\epsilon^2}\mathbf{Z}_j^\top\mathbf{Z}_j\right)}\right) + \frac{1}{2}\cdot\log\beta.
\end{aligned}$$

The $\beta = \sin\{R(\tilde{\mathbf{Z}}_1), R(\tilde{\mathbf{Z}}_2)\}$ means the principal angle of the $\tilde{\mathbf{Z}}_1, \tilde{\mathbf{Z}}_2$, which measures the difference of subspaces. Maximizing β is to maximize the difference of the subspace. According to Corollary 1, we prove that the $\mathbf{z}_*$ in $\mathbf{Z}$ are pairwise orthogonal, then the $\tilde{\mathbf{z}}_*$ in $\tilde{\mathbf{Z}}$ will also be pairwise orthogonal. So the maximum value of the product of principal angle sines between different subspaces of $\mathbf{Z}$ and $\tilde{\mathbf{Z}}$ are equal to 1. And then they reach the maximum at the same time.

B EXPERIMENTAL SETTING

To reproduce the results of the proposed method, we provide the details of training, dataset, baselines.

B.1 Training Setting

G^2R is implemented using PyTorch 1.7.1 [28] and PyTorch Geometric 1.6.3 [8]. All models are initialized with Xavier [11] initialization, and are trained with Adam [17] optimizer. For linear evaluation mode for node classification, we use the existing implementation of logistic regression with L_2 regularization from Scikit-learn [2]. For all datasets and baselines, we perform experiments five times with different seeds and report the mean and standard deviation of accuracies (%) for node classification. In training phase, we set the dimension of node representations as 512. We perform grid search on the number of epoch and learning rate. For unsupervised baselines, we use the public code released by the authors. All experiments are conducted on a Linux server with two AMD EPYC 7282 CPUs and four NVIDIA RTX3090 GPUs (24GB memory each).

B.2 Dataset

Following the previous works, we use eight benchmark datasets to evaluate G^2R and baselines, including Cora, CiteSeer, PubMed, CoraFull, Coauthor CS, Coauthor Physics, Amazon Computers, and Amazon Photo [1, 22, 33, 42]. All datasets used throughout

experiments are available in PyTorch Geometric [8] libraries. The details of the dataset are as follows:

- **Planetoid** [42]. Planetoid dataset includes Cora, CiteSeer and PubMed, which is representative citation network datasets. These datasets contains a number of machine learning papers, where nodes and edges denote documents and citation, respectively. Node features are bay-of-words for documents. Class labels indicate the field of documents.
- **CoraFull** [1] is a well-known citation network that contains labels based on the paper topic. This dataset is additionally extracted from the original data of the entire network of Cora. Specifically, CoraFull contains the entire citation network of Cora, while the Planetoid Cora dataset is its subset.
- **Coauthor** [33]. Coauthor Physics is co-authorship graph based on the Microsoft Academic Graph from the KDD Cup 2016 challenge. Nodes are authors and edges indicate co-authored a paper. Node features represent paper keywords for each author's papers, and class labels indicate the most active fields of study for each author.
- **Amazon** [33]. Amazon dataset includes Computers and Photo which are extracted from co-purchase graph [22]. Nodes represent goods, edges indicate that two goods were bought together. The node features are bag-of-words encoded product reviews and class labels are the product category.

B.3 Baselines for unsupervised learning

We list the baselines used for the unsupervised learning comparison.

- **Features**. We use the original feature as input.
- **PCA** [39] and **SVD** [12]. These two methods are matrix decomposition based methods and only contain the node features information. We use the node features after PCA(or SVD) dimensionality reduction as the input feature.
- **LLE** [32] and **Isomap** [35]. These two methods are manifold based dimensionality reduction methods and only contain the node features information. We use the reduced node feature as the node representations.
- **DGI** [6] [38] is a general approach for learning node representations within graph-structured data in an unsupervised manner, which relies on maximizing mutual information between patch representations and corresponding high-level summaries of graphs—both.
- **GraphCL** [7] [44] is a graph contrastive learning framework for learning unsupervised representations of graph data with graph data augmentations.
- **GRACE** [8] [44] is an unsupervised graph representation learning method. GRACE first generates two views of graph by corruption and then maximizes the agreement of node representations in these two views.
- **GMI** [9] [29] measures the correlation between input graphs and high-level hidden representations. GMI directly maximizes the mutual information between the input and output of a graph encoder in terms of node features and topological structure.

B.4 Baselines for supervised learning

We mainly adopt supervised GNN models as baselines for supervised learning comparison. In addition to GNN models, we also consider the following baselines: Logistic Regression (LogReg), Multi-Layer Perceptron (MLP), Label Propagation (LabelProp) and Normalized Laplacian Label Propagation (LabelProp NL). Then details of baseline models are listed as follows:

- **MLP** uses the node features as input and the node labels as output, which only leverages the node feature information while ignores the connection information.
- **LabelProp** [47] uses unlabeled data to help labeled data in classification. Labels were propagated with a combination of random walk and clamping. LabelProp only considers the graph structure.
- **GCN** [18] Graph Convolutional Network is one of the earlier models that works by performing a linear approximation to spectral graph convolutions.
- **MoNet** [25] generalizes the GCN architecture and allows to learn adaptive convolution filters.
- **GAT** [37] proposes masked self-attentional layers that allow weighing nodes in the neighborhood differently during the aggregation step, which overcomes the shortcomings of prior GNN methods by approximating the convolution.
- **SAGE** [14]. GraphSAGE focuses on inductive node classification but can also be applied for transductive settings.
- **ChebNet** [7]. ChebNet is a formulation of CNNs concerning spectral graph theory, which provides the necessary mathematical background and efficient numerical schemes to design fast localized convolutional filters on graphs.
- **SGC** [40]. Simple Graph Convolution (SGC) reduces the excess complexity of GCNs by repeatedly removing the nonlinearities between GCN layers and collapsing the resulting function into a single linear transformation.
- **APPNP** [19]. Approximate personalized propagation of neural predictions (APPNP) is a fast approximation to personalized propagation of neural predictions (PPNP), which utilizes this propagation procedure to construct a simple model. APPNP leverages a large, adjustable neighborhood for classification and can be easily combined with any neural network.

C METRICS OF COMMUNITY DETECTION

The metrics are implemented by https://networkx.org. The *modularity* is defined as $Q = \sum_{c=1}^{n} \left[\frac{L_c}{m} - \gamma \left(\frac{k_c}{2m} \right)^2 \right]$, where the sum iterates over all communities c, m is the number of edges, L_c is the number of intra-community links for community c, k_c is the sum of degrees of the nodes in community c, and γ is the resolution parameter; The *coverage* of a partition is the ratio of the number of intra-community edges to the total number of edges; The *performance* of a partition is the number of intra-community edges plus inter-community non-edges divided by the total number of potential edges.

[6] https://github.com/rusty1s/pytorch_geometric/blob/master/examples/infomax_inductive.py
[7] https://github.com/Shen-Lab/GraphCL
[8] https://github.com/CRIPAC-DIG/GRACE
[9] https://github.com/zpeng27/GMI